

PhD Notes

From 21 July to 15 August

Zhihao Li

July 30, 2025

Contents

I 2025 Fall: Incoming PhD

1 July

July

1	July 21	
	1.1	Mixture-of-Experts (MoE) . .
	1.2	Vim
2	July 22	
	2.1	Vim
3	July 23	
	3.1	Self-Attention Coding

4	July 24		9
	4.1	Dot Product	9
	4.2	Linear Algebra	9
	4.3	Contrasive Learning	10
	4.4	Next-X Autoregressive (XAR)	11
5	July 25		12
	5.1	Linear Algebra	12
6	July 27		13
	6.1	Multimodal Large Language Models(MLLMs)	13
7	July 29		17
	7.1	Finetuning Pretrained Trans- formers	17
8	July 30		23
	8.1	Inequality	23
3	Appendices		24
3	Appendix A Formulas		24
4	1 Gaussian distribution		24
5	Bibliography		25
6	Alphabetical Index		26

Part I

2025 Fall: Incoming PhD

Chapter 1

July

Contents

1	July 21	3
1.1	Mixture-of-Experts (MoE)	3
1.2	Vim	4
2	July 22	5
2.1	Vim	5
3	July 23	6
3.1	Self-Attention Coding	6
4	July 24	9
4.1	Dot Product	9
4.2	Linear Algebra	9
4.3	Contrastive Learning	10
4.4	Next-X Autoregressive (XAR)	11
5	July 25	12
5.1	Linear Algebra	12
6	July 27	13
6.1	Multimodal Large Language Models (MLLMs)	13
7	July 29	17
7.1	Finetuning Pretrained Transformers	17
8	July 30	23
8.1	Inequality	23

1 July 21

1.1 Mixture-of-Experts (MoE)

GLaM: Efficient Scaling of Language Models with Mixture-of-Experts

Du et al. 2022 将 MoE 思想引入到 Transformer 结构中解决模型随数据量的缩放问题。
作为一种大型稀疏网络，利用稀疏激活的混合专家模式，能够扩大模型容量同时降低计算资源。

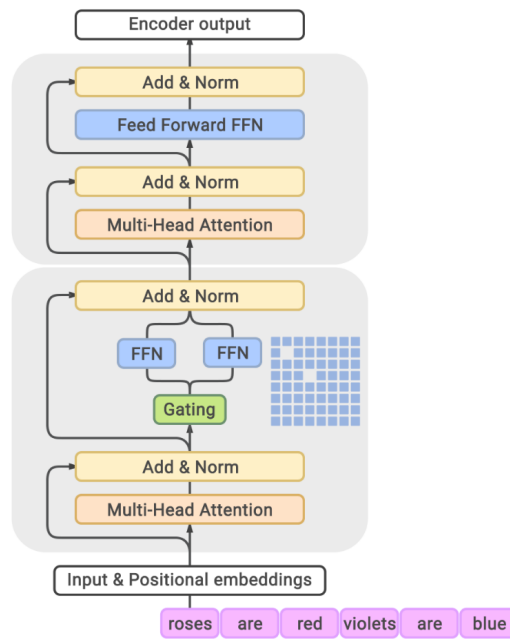


Figure 1.1: GLaM model architecture

Gating Function 门控函数 (Gating Function) 使用一个 Softmax 激活函数建模在多个专家之间的概率分布，这决定了哪一个专家最适合处理当前的输入。

Sparsely Activation 每一个 MoE 网络层尽管有许多的参数，但各个专家都是稀疏地被激活，保证了模型只有一个专家子集被利用。

1.2 Vim

I just learn the Vim from scratch using the well designed website [vimhero](https://vimhero.com/) as the tutorial.

Modes

Normal Mode

- Moving the cursor
- Deleting characters, words, lines, etc
- Performing operations like Copy / Paste

Insert Mode Edit and insert text.

Normal Mode

Mouse Position Movement In the **Normal Mode**, use the **h j k l** keys to move one character in any direction.

- h: move left

- j: move down
- k: move up
- l: move right

Word Movement We can move the current position word by word.

- w: move to the next word
- e: move to the end of a word
- b: move back a word

Insert Mode

There are two ways to insert the character according to the insert position.

- i: enter insert mode before your cursor. Just insert the character before the cursor.
- a: enter insert mode after your cursor. Just insert the character after the cursor.

Insert one character at the beginning and end of a line. **The following two usages are just running at the normal mode and then it will enter the insert mode.**

- I: enter insert mode at the beginning of a line.
- A: enter insert mode at the end of a line.

2 July 22

2.1 Vim

Opening New Lines

We can use the **open** operator to add a new line above or below your cursor and then enter insert mode on that line.

- o: open a new line **below the cursor**.
- O: open a new line **above the cursor**.

Making Small Edits

Make small edits to text such as delete, replace.

- x: delete the character **under the cursor**
- s: delete the character under the cursor and **enter insert mode**
- r: replace the character under the cursor with the next character you type

Movement Block by Block

Moving the cursor can go block by block that it treats everything between spaces as one chunk.

- **W**: move to the next WORD (cross one block)
- **E**: move to the end of a WORD
- **B**: move back a WORD

Moving to Line Ends

Just using the following keys to move the cursor to the beginning or end of a line without entering insert mode.

- **0**: move to the beginning of a line
- **\$**: move to the end of a line
- **_**: move to the first word in a line

Find Motion

Use the find motions to quickly jump to a specific character **within a line**.

- **f char**: move forward to the next occurrence of {char} within the line.
- **F char**: move backward to the previous occurrence of {char} within the line.
- **;**: repeat the last find motion that just continues to find the next character.

3 July 23

3.1 Self-Attention Coding

Self-Attention Mechanism

The self-attention mechanism is a fundamental concept in transformer models and is widely used in natural language processing (NLP) and computer vision (CV). It allows models to dynamically weigh different parts of the input sequence, enabling them to capture long-range dependencies effectively.

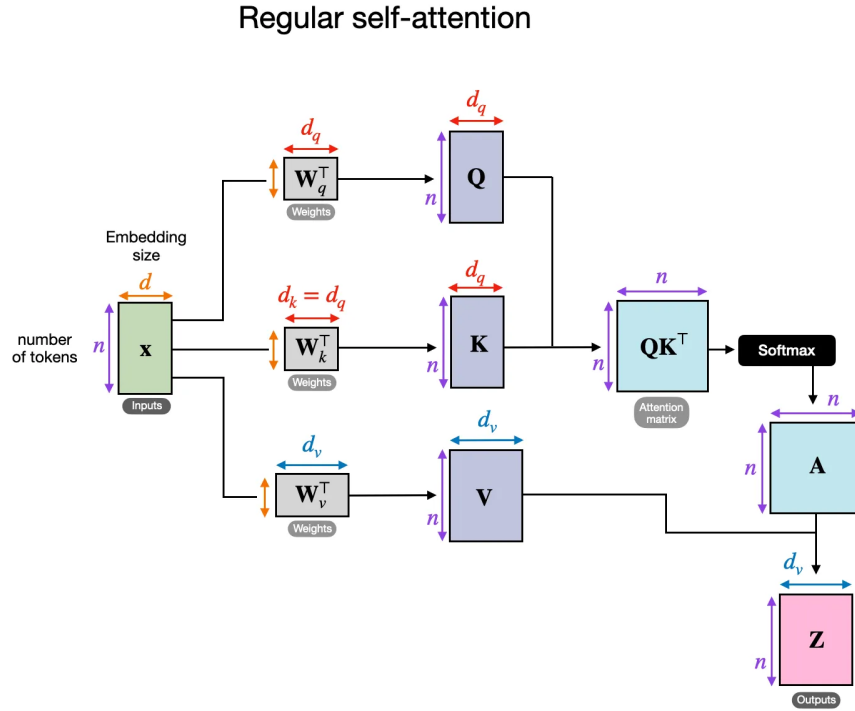


Figure 1.2: Self-Attention architecture

Understanding Self-Attention

Self-attention helps a model determine **which parts of an input sequence are relevant to each other**. Instead of treating every word or token equally, self-attention assigns different weights to different parts of the sequence, allowing the model to capture contextual relationships.

For example, in machine translation, self-attention allows the model to **focus on relevant words** from the input sentence when generating each word in the output.

Mathematical Formulation of Self-Attention

Given an input sequence X , self-attention computes three key components:

1. **Query(Q)**: Represents the current token we are processing.
2. **Key(K)**: Represents each token in the sequence.
3. **Value(V)**: Contains the actual token embeddings.

The Query, Key and Value matrices are computed as:

$$Q = XW_Q, K = XW_K, V = XW_V.$$

where W_Q, W_K, W_V are learned weight matrices.

The attention scores are computed using the **scaled dot-product attention**:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

where d_k is the dimensionality of the key vectors.

Why Self-Attention is Powerful?

Captures long-range dependencies Unlike RNNs, which process input sequentially, self-attention can relate any word in the sequence to any other word, regardless of distance.

Parallelization Since self-attention is computed simultaneously across the entire sequence, it is much faster than sequential models like LSTMs.

Contextual Understanding Each token is contextually enriched by attending to relevant tokens in the sequence.

Applications of Self-Attention

Self-attention is widely used in:

1. **Transformer models (e.g., BERT, GPT-3)** for language modeling.
2. **Speech processing models** for transcribing audio.
3. **Vision Transformers (ViTs)** for computer vision tasks.
4. **Recommender systems** for learning item-user relationships.

How to Code

```

1  import numpy as np
2
3  # define some functions
4  def compute_qkv(X, W_q, W_k, W_v):
5      Q = np.dot(X, W_q)
6      K = np.dot(X, W_k)
7      V = np.dot(X, W_v)
8      return Q, K, V
9
10 def self_attention(Q, K, V):
11     d_k = Q.shape[1]
12     scores = np.matmul(Q, K.T) / np.sqrt(d_k)
13     attention_weights = np.exp(scores) / np.sum(np.exp(scores), axis=1, keepdims=True)
14     attention_output = np.matmul(attention_weights, V)
15     return attention_output
16
17 X = np.array([[1, 0], [0, 1]])
18 W_q = np.array([[1, 0], [0, 1]])
19 W_k = np.array([[1, 0], [0, 1]])
20 W_v = np.array([[1, 2], [3, 4]])
21
22 Q, K, V = compute_qkv(X, W_q, W_k, W_v)
23 output = self_attention(Q, K, V)
24
25 print(output)
26
27 [[1.6604769 2.6604769]
28  [2.3395231 3.3395231]]

```

4 July 24

4.1 Dot Product

点积又称为数量级：逐元素相乘再求和。

Vector-Vector Dot Product 假设有两个向量 $\vec{a} = [a_1, a_2, \dots, a_n]$, $\vec{b} = [b_1, b_2, \dots, b_n]$, 则向量之间的点积为：

$$\vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i.$$

在 Python 中实现可以利用 Numpy 中的 `np.dot` 计算。

```
1 import numpy as np
2 dot_result = np.dot(a, b)
```

Matrix-Vector Dot Product 当计算矩阵与向量之间的点积时，类似矩阵乘法运算，矩阵的列数应当等于向量的长度。实际上矩阵乘法运算，就是多个向量之间的点积运算，每一个输出值都是两个向量之间的相似性度量。矩阵乘法反映了线性变换的复合关系，而点积的重复应用则体现了特征在不同基方向上的投影过程。

给定一个矩阵 $A \in \mathbb{R}^{m \times n}$ 与一个向量 $\vec{x} \in \mathbb{R}^n$, 则点积结果 $\vec{y} = A\vec{x} \in \mathbb{R}^m$, 其中：

$$y_i = \vec{a}_i \cdot \vec{x} = \sum_{j=1}^n A_{ij} x_j.$$

Python 中仍然可以利用 `np.dot` 直接实现：

```
1 import numpy as np
2 A = np.array([[1, 2, 3],
3               [4, 5, 6]])
4 x = np.array([0.1, 0.2, 0.3])
5 dot_result = np.dot(A, x)
```

4.2 Linear Algebra

列表矩阵转置 对于二维列表实现转置操作，高效的做法可以利用 `zip` 拉链操作。`zip` 会将引用的 `a` 列表矩阵按列组合。

```
1 def transpose_matrix(a: list[list[int|float]]) -> list[list[int|float]]:
2     return [list(i) for i in zip(*a)]
3     # or
4     # list(map(list, zip(*a)))
```

矩阵特征值计算 对于二维矩阵计算其特征值以及特征向量，并按照特征值递减顺序排列。

只有方阵 $n \times n$ 才有特征值，非方阵没有特征值，可以用奇异值分解 (SVD)。

在这个问题上，理解矩阵在几何层面本质上是一次**空间的线性变换（缩放 + 旋转）**，特征值和特征向量的存在是反映空间变换中存在某个向量变换前和变换后处于同一直线上的关系，即只进行了缩放。

$$A\vec{x} = \lambda\vec{x}.$$

其中， A 是方阵， $\vec{x}$ 是特征向量， λ 是对应的特征值。特征值 λ 是一个标量，表示在特征向量 $\vec{x}$ 上的缩放因子。那么，求解特征值和特征向量的过程可以通过求解特征方程：

$$(A - \lambda I)\vec{x} = 0.$$

其中， I 是单位矩阵。通过求解这个方程，可以得到特征值 λ ，然后将其代入原方程来求解对应的特征向量 $\vec{x}$ 。

```
1 import numpy as np
2 eigenvalues, eigenvectors = np.linalg.eig(matrix)
3
4 # sort by eigenvalues in descending order
5 sorted_indices = np.argsort(eigenvalues)[::-1]
6 sorted_eigenvalues = eigenvalues[sorted_indices]
7 sorted_eigenvectors = eigenvectors[:, sorted_indices]
```

4.3 Contrastive Learning

对比学习的目标是**学习一个嵌入空间，使得相似的样本彼此紧邻，而差异的样本则相互远离**。这种表示学习方法可以应用在监督和无监督学习任务中。处理无监督数据时，对比学习是自监督学习中最成功的方法之一。

Contrastive Loss

对比损失 (Chopra et al. 2005) (Contrastive Loss) 是被用于对比学习范式的最早训练目标之一。

给定一组输入样本 x_i ，每一个样本都对应一个标签 $y_i \in \{1, \dots, L\}$ ，其中 L 是总的类别。希望学习一个函数 $f_\theta(\cdot) : \mathcal{X} \rightarrow \mathbb{R}^d$ 编码 x_i 为一个嵌入向量使得来自同一个类别的样本具有相似的嵌入向量，而来自不同类别的样本则具有不同的嵌入向量。

因此，对比损失定义为：接收一组样本输入 (x_i, x_j) ，以**最小化相同类别之间的嵌入距离、最大化不同类别之间的嵌入距离**。

$$\mathcal{L}_{\text{cont}}(x_i, x_j, \theta) = \mathbb{1}_{y_i \neq y_j} \|f_\theta(x_i) - f_\theta(x_j)\|_2^2 + \mathbb{1}_{y_i = y_j} \max(0, \epsilon - \|f_\theta(x_i) - f_\theta(x_j)\|_2)^2.$$

其中， ϵ 是一个超参数，定义不同类别样本之间距离的最小下界。

Triplet Loss

三元组损失 (Triplet Loss) 是对比学习的另一种常用损失函数，最初被 FaceNet(Schroff et al. 2015) 提出，用于解决不同姿势和角度下相同人物的人脸识别问题。

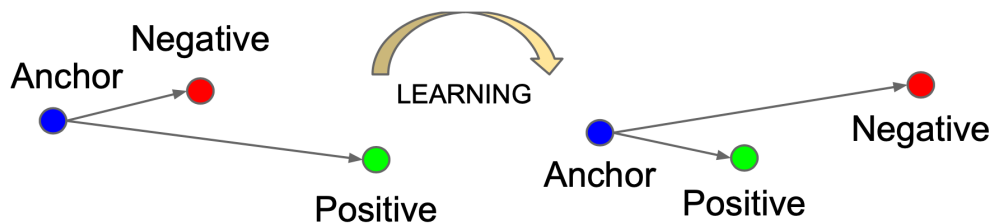


Figure 1.3: 三元组损失示意图 (Schroff et al. 2015)

给定一个锚点输入 x ，并选择一个与锚点来自同一类别的正样本 x^+ 和一个来自不同类别负样本 x^- ，三元组损失函数的目标是最小化锚点与正样本之间的距离同时最大化锚点与负样本之间的距离。

$$\mathcal{L}_{\text{triplet}}(x, x^+, x^-) = \sum_{x \in \mathcal{X}} \max(0, \|f_{\theta}(x) - f_{\theta}(x^+)\|_2^2 - \|f_{\theta}(x) - f_{\theta}(x^-)\|_2^2 + \epsilon).$$

其中，边际参数 (margin parameter) ϵ 定义为相似对与不相似对之间的最小距离差异。选择合适的 x^- 是提高模型能力的关键。

4.4 Next-X Autoregressive (XAR)

Beyond Next-Token: Next-X Prediction for Autoregressive Visual Generation

论文提出了 xAR 模型，旨在统一自回归视觉生成范式，采取了一种通用的 Next-X 预测框架，其中 X 是一种灵活的预测实体，可以表示一个独立图像块 (token)，一个单元 (一组空间连续的 tokens)，一个子样本 (非局部组)，一整个图像 (flow-matching)，一个尺度 (粗到细分辨率)，如图1.4所示。

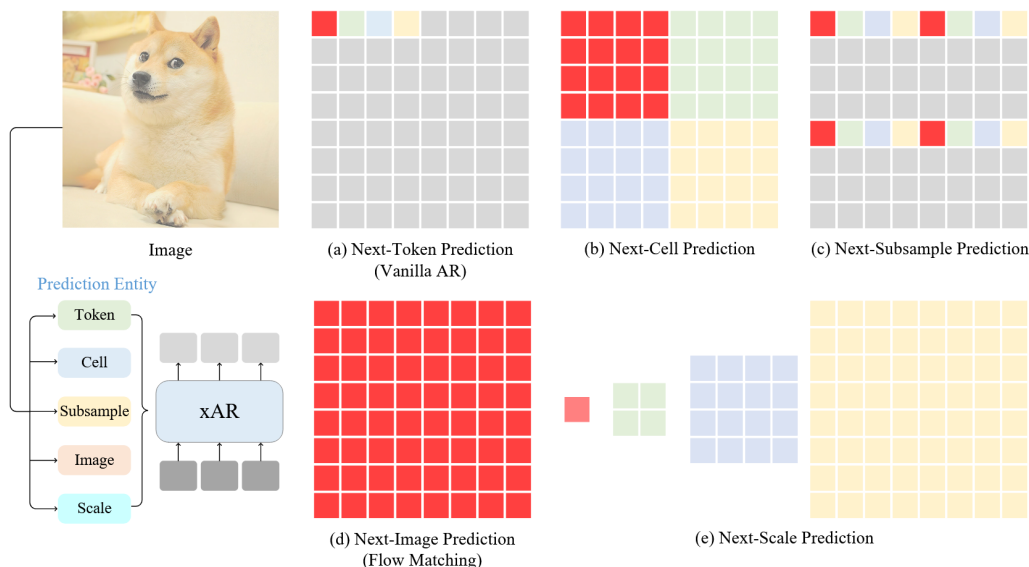


Figure 1.4: Visual Generation(Sucheng et al. 2025)

研究动机 当前 Teacher forcing 自回归生成方法中在每一步将之前步骤的 ground truth 作为输入而不是模型自身预测输出，尽管这种方法能够提高训练稳定性但会引入额外的偏差。在推理的时候，没有 ground truth 监督仅仅依赖过去的预测输出，误差会随着时间的推移而累积，导致生成质量下降，如图1.5所示。

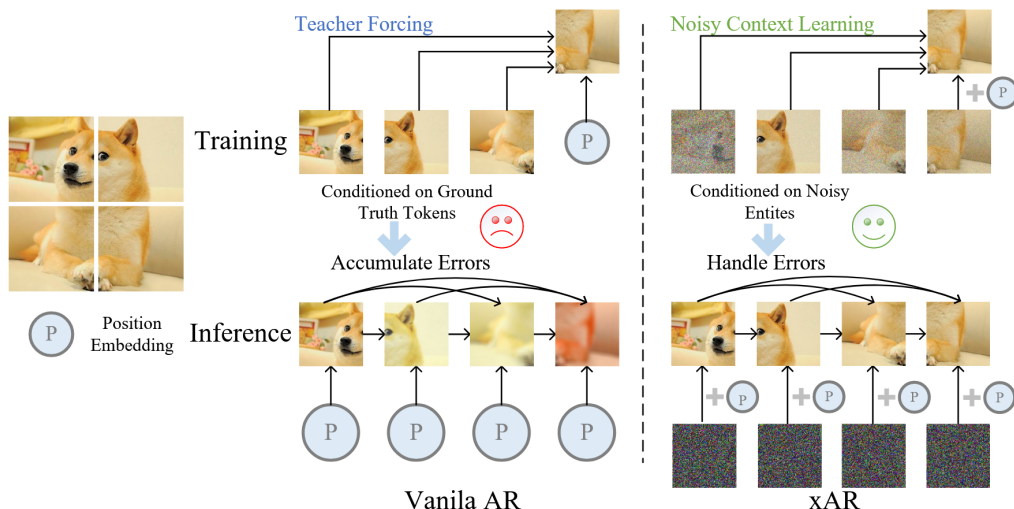


Figure 1.5: Conditioning Mechanism Comparison between Vanila AR vs. xAR(Sucheng et al. 2025)

核心创新 xAR 通过在**每一步的 token 上添加噪声**来学习不完美、损坏的或是部分不精确的条件。通过 flow-matching 机制，xAR 将离散的 token 分类任务（codebook 学习的 token 表示）转换为一个连续的实体回归任务。

5 July 25

5.1 Linear Algebra

矩阵求逆 判断矩阵是否可逆，只要求矩阵的秩是否为 0 即可。矩阵的逆可以用 NumPy 库实现，并且 np.dot 对于二维矩阵而言，点积结果就是矩阵乘法。

```

1 import numpy as np
2
3 def transform_matrix(A: list[list[int|float]], T: list[list[int|float]], S:
  ↳ list[list[int|float]]) -> list[list[int|float]]:
4     if np.linalg.det(T) == 0 or np.linalg.det(S) == 0:
5         return -1
6
7     transformed_matrix = np.linalg.inv(T) @ A @ S
8     # or
9     # transformed_matrix = np.matmul(np.linalg.inv(T), A) @ S
10    # transformed_matrix = np.dot(np.linalg.inv(T), A) @ S
11    return transformed_matrix

```

协方差矩阵 对于一个矩阵，每行是一个特征，每列是一个样本，协方差矩阵是求解每个特征之间的协方差。

$$\text{cov}(X, Y) = \frac{\sum_{k=1}^m (X_k - \bar{X})(Y_k - \bar{Y})}{m - 1}.$$

根据公式可以直接求解协方差矩阵：

```

1 def calculate_covariance_matrix(vectors: list[list[float]]) -> list[list[float]]:
2     n_features = len(vectors)
3     n_observations = len(vectors[0])
4     covariance_matrix = [[0 for _ in range(n_features)] for _ in range(n_features)]
5
6     means = [sum(feature) / n_observations for feature in vectors]
7
8     for i in range(n_features):
9         for j in range(i, n_features):
10             covariance = sum((vectors[i][k] - means[i]) * (vectors[j][k] - means[j]) for k
11                             ↪ in range(n_observations)) / (n_observations - 1)
12             covariance_matrix[i][j] = covariance_matrix[j][i] = covariance
13
14     return covariance_matrix

```

当然，利用 NumPy 库可以直接计算协方差矩阵：

```

1 import numpy as np
2 def calculate_covariance_matrix(vectors: list[list[float]]) -> list[list[float]]:
3     cov_mat = np.cov(np.array(vectors)).tolist()
4     return cov_mat

```

6 July 27

6.1 Multimodal Large Language Models(MLLMs)

多模态大语言模型是指能够处理多种类型输入的大语言模型，每一种模态都代表着一种类型数据（文本、音频、图像、视频等）。

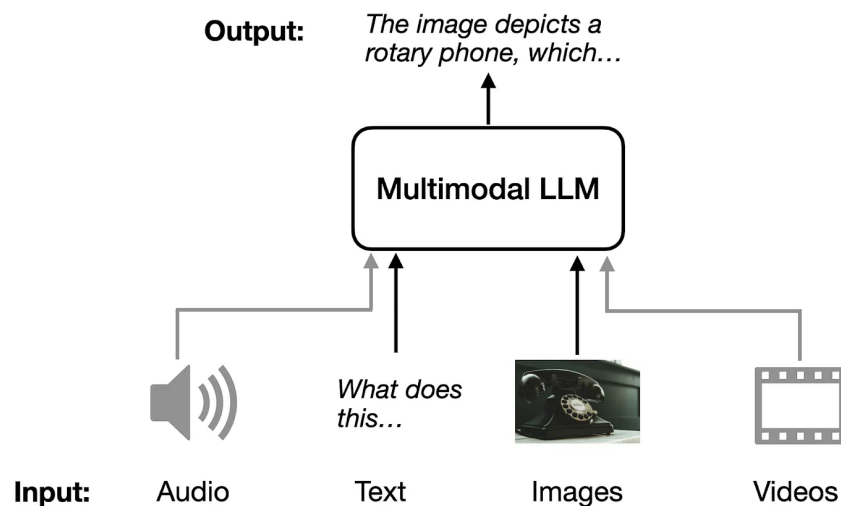


Figure 1.6: Multimodal Large Language Models(Raschka 2024a)

Building Mechanisms

现有两种方式构造 MLLMs，一种是基于解码器结构，另外一种是基于交叉注意力机制。

1. **Decoder-based Architecture**: 统一的嵌入解码器结构。
2. **Cross-Attention-based Architecture**: 交叉注意力机制结构。

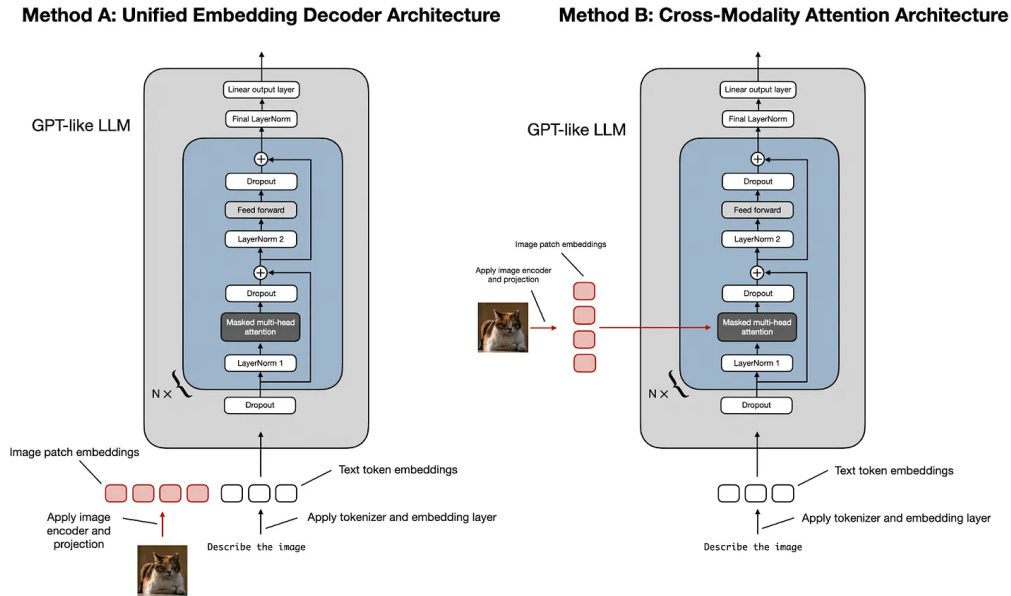


Figure 1.7: 构造 MLLMs 的两种方式 (Raschka 2024a)

解码器结构的 MLLMs 通过将不同模态的输入嵌入到统一的解码器中来处理多模态数据，允许 LLMs 通过拼接各个模态的特征嵌入，统一进行解码。

交叉注意力机制的 MLLMs 通过在注意力层利用交叉注意力机制直接融合图像和文本嵌入表示。

Decoder-based Architecture

文本处理通过 Tokenizer 将文本统一转换为特征嵌入表示，而对于图像通常采取 ViT 结构的编码器进行处理，例如 (CLIP, OpenCLIP)。

其中，ViT 结构中的线性投影层一般采用一个单一的线性层（全连接层），将图像块特征投影到与 Transformer 编码器兼容的大小。

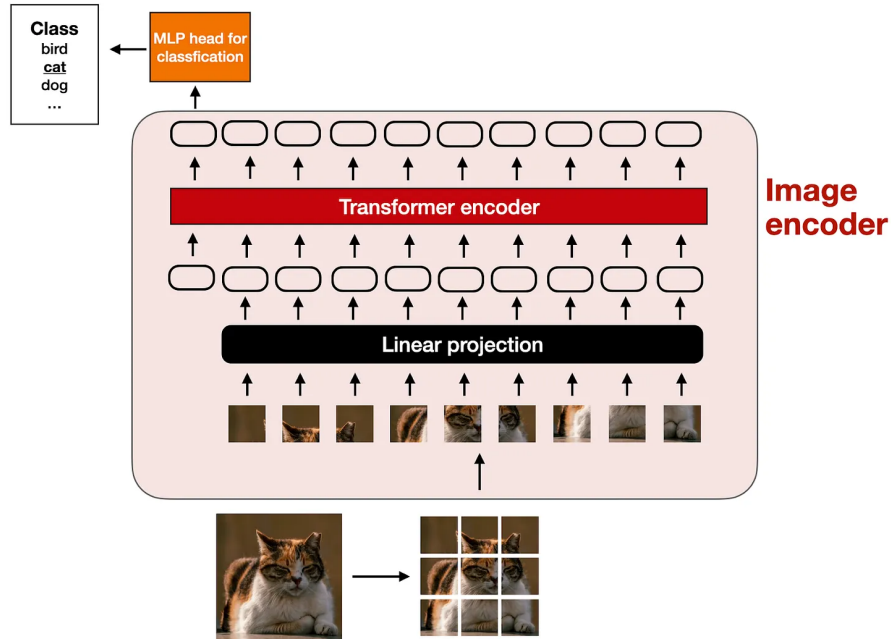


Figure 1.8: 经典 ViT 结构 (Raschka 2024a)

```

1 import torch
2
3 class PatchProjectionLayer(torch.nn.Module):
4
5     def __init__(self, patch_size, num_channels, embedding_dim):
6         super().__init__()
7         self.patch_size = patch_size
8         self.num_channels = num_channels
9         self.embedding_dim = embedding_dim
10        self.projection = torch.nn.Linear(
11            patch_size * patch_size * num_channels, embedding_dim
12        )
13
14    def forward(self, x):
15
16        batch_size, num_patches, channels, height, width = x.size()
17        x = x.view(batch_size, num_patches, -1) # Flatten each patch
18        x = self.projection(x) # Project each flattened patch
19        return x
20
21
22 # Example Usage:
23 batch_size = 1
24 num_patches = 9 # Total patches per image
25 patch_size = 16 # 16x16 pixels per patch
26 num_channels = 3 # RGB image
27 embedding_dim = 768 # Size of the embedding vector
28
29 projection_layer = PatchProjectionLayer(patch_size, num_channels, embedding_dim)
30
31 patches = torch.rand(

```



```

32     batch_size, num_patches, num_channels, patch_size, patch_size
33 )
34
35 projected_embeddings = projection_layer(patches)
36 print(projected_embeddings.shape)
37
38 # This prints
39 # torch.Size([1, 9, 768])

```

如图1.9所示，图像 Tokenizer 在图像编码器后增加了一个额外的 **projector** 模块，也被称为 **adapter**, **adaptor**, **connector**。它主要是另外一个线性投影层，目的是将图像编码器的输出与文本特征嵌入进行维度对齐，使得可以直接将文本和图像特征一起拼接传到 Transformer 处理。

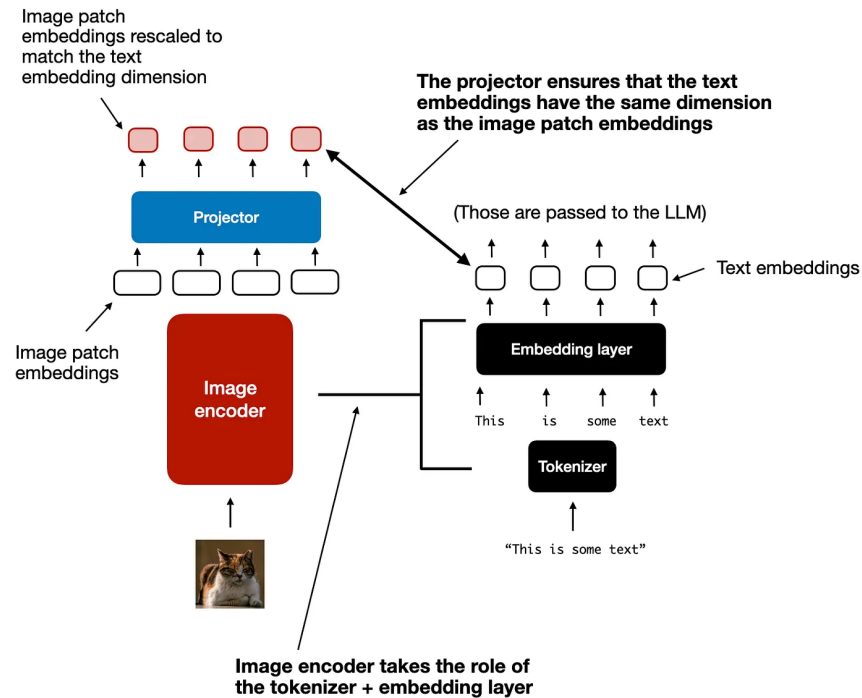


Figure 1.9: 图像和文本 Tokenizer 的比较 (Raschka 2024a)

此外，还有研究方法直接利用一个线性投影层学习的图像块嵌入表示作为 LLMs 的图像输入部分，而不依赖额外的预训练编码器，以简化模型结构和训练设置。

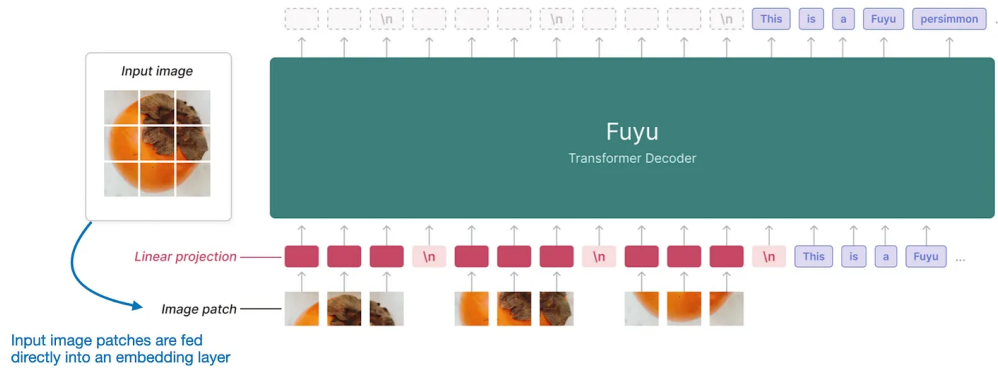


Figure 1.10: Fuyu 多模态大语言模型结构 (Bavishi 2023)

Cross-Attention-based Architecture

基于交叉注意力机制的 MLLMs 通过在 Transformer 的注意力层中引入交叉注意力机制来处理多模态数据。

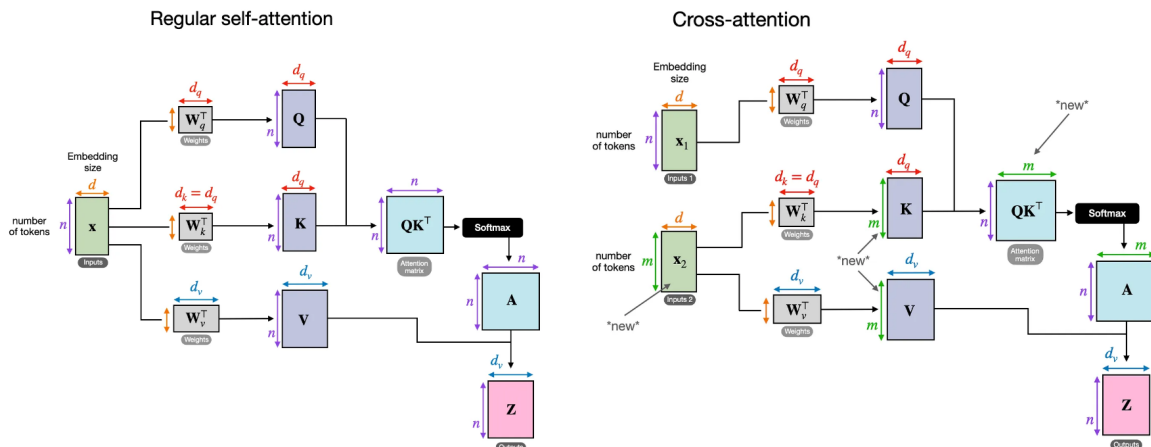


Figure 1.11: 自注意力机制与交叉注意力机制结构 (Raschka 2024a)

MLLMs 一般以一个预训练且指令微调的纯文本 LLM 作为基础模型进行训练。

- Image Encoder: 通常选择 CLIP 并且在整个训练过程中保持不变；
- Projector: 一般只训练这一部分，并固定 LLM frozen，有时考虑到 Projector 学习能力不足，会将 LLM 几层也进行微调；
- Cross-Attention: 交叉注意力机制的 MLLMs 通过对应的交叉注意力层需要在整个过程中进行训练。

7 July 29

7.1 Finetuning Pretrained Transformers

Transformer 的堆叠构成了大模型的基本框架，大模型微调技术是指在预训练的 Transformer 模型基础上进行特定任务的微调，以适应不同的应用场景，主要包括基于特征的方法，上下文提示和更新部分

参数三种方法。

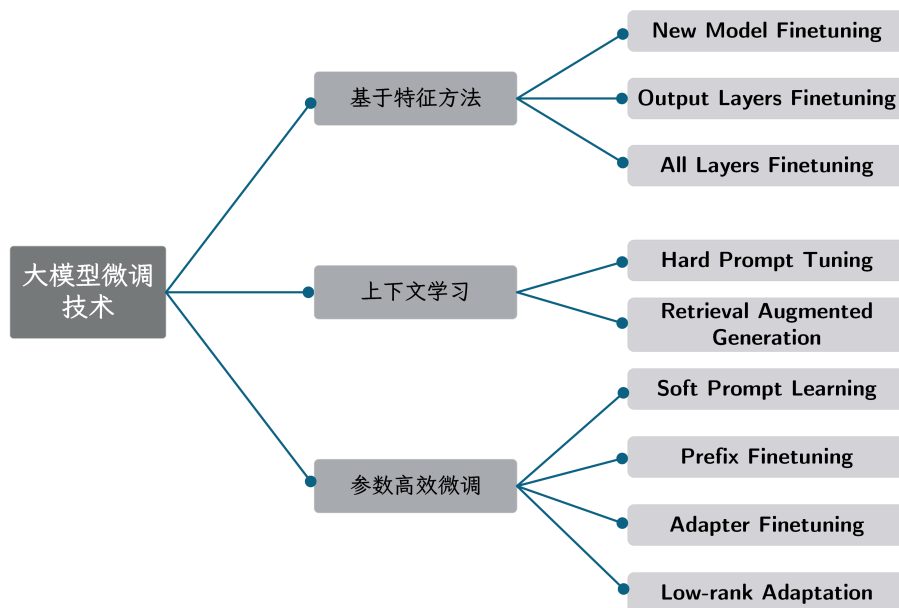


Figure 1.12: 大模型微调技术概览

Feature-based Methods

基于特征的方法是指通过预训练模型提取嵌入特征表示（embeddings），然后训练一个新的下游模型，例如线性分类器（Linear Classifier）或支持向量机（SVM）等。

如图1.13所示，根据微调的模型包括下游模型、全连接层的不同以及更新参数不同，主要分为以下三种基于特征的微调方法。

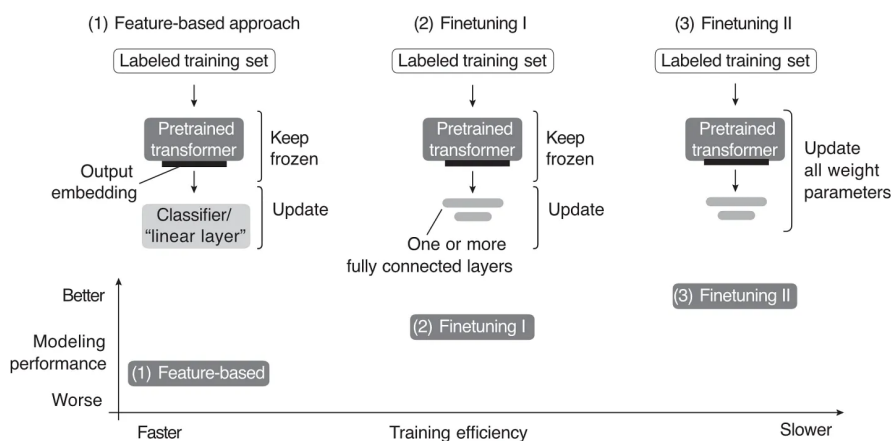


Figure 1.13: 三种常见的大模型微调技术 (Raschka 2024b)

Feature-based 这种基于特征的微调方法，可以认为是基于模型的微调方法：通过将 LLM 的骨干网络参数固定，当作一个特征提取器，并在提取的特征嵌入上，训练一个新的下游模型。

这种微调方法旨在学习新添加的下游模型，而不改变预训练模型的参数，是非常高效的学习方法。

Finetuning-based 微调大模型主要应用在下游任务上，最常见的做法就是微调大模型的输出层，只更新输出层参数（Finetuning I），或更新所有层参数（Finetuning II）。

预训练大模型中通常会有线性输出层，主要将高层语义特征进行线性变换，映射到下游任务的标签空间。因此，对于不同的下游任务，可以通过微调线性输出层来适应不同的任务，甚至可以额外增加若干线性层以增加微调模型的表达能力。

为了实现更高的准确性，也可以微调大模型的所有层参数（Finetuning II），但这种方式就非常耗费计算资源和时间。

In-Context Learning Methods

上下文学习 (In-Context Learning) 旨在在输入或提示中提供语境或任务示例，使得模型能够推断出渴望的行为并生成恰当的回答。

这种方法旨在构造大模型需要的提示信息，而不微调任何的模型参数，目的在于优化提示本身，效果往往不如 Feature-based 类方法。

Hard Prompt Tuning 硬提示微调（Hard Prompt Tuning）是指输入 Token 具有不可微分的特性，不能够通过学习优化算法进行优化。这一类方法并不修改模型任何参数，旨在通过一个小型有标签数据集识别出特定下游任务下最好的提示范式，其中提示范式往往是组合式的提示词，如下所示。

硬提示学习过程需要人力参数比较不同提示词的质量，是非常耗费人力成本的。

```

1 "Translate the German sentence '{german_sentence}' into English: {english_translation}"
2
3 "German: '{german_sentence}' | English: {english_translation}"
4
5 "From German to English: '{german_sentence}' -> {english_translation}"

```

LLM Indexing 利用上下文学习的另一类方法是 LLM 索引（LLM Indexing），旨在通过将输入文本转换为向量索引来优化提示信息。

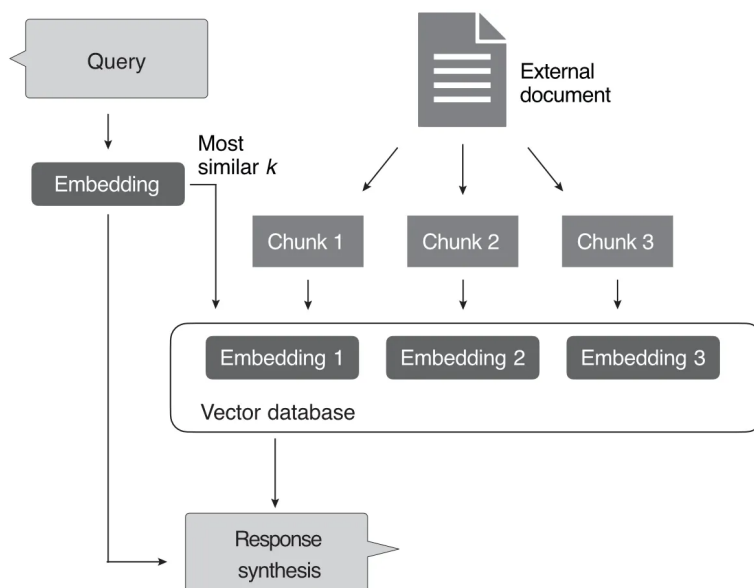


Figure 1.14: LLM Indexing 从外部文档中进行信息检索 (Raschka 2024b)

如图1.14所示，一个信息检索系统将一个文本或网站解析为多个小块（chunk），并将这些小块转换为嵌入向量，存储在向量数据库中。当用户查询时，检索模块计算查询向量与数据库中的嵌入向量之间的相似度，并选择前 k 个最相似的嵌入向量作为检索结果，注入到提示中合成最后的回答。

因此，LLM Indexing 也被称为连接 LLM 与现有数据源的框架/过程，其中的一个示例是检索增强生成（Retrieval Augmented Generation, RAG），它通过检索相关文档来增强生成模型的回答。

Parameter-Efficient Finetuning Methods

相比于之前的方法而言，都不够高调地微调大模型方法，最近几年针对性地提出了一些更加高效的参数微调方法。

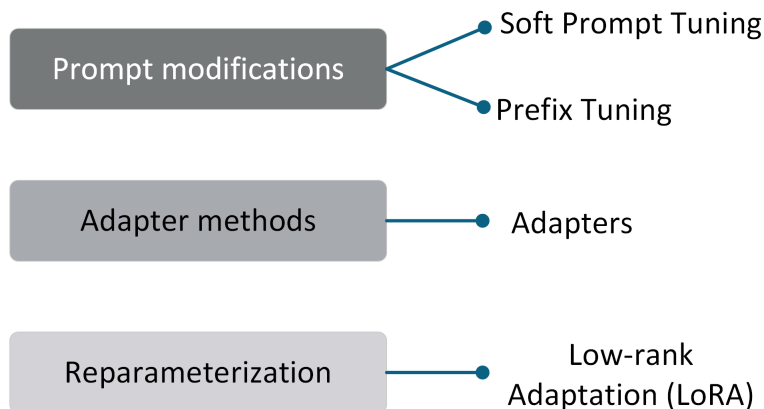


Figure 1.15: 参数高效微调方法

Soft Prompt Tuning 软提示学习（Soft Prompt Tuning）是一种通过优化提示词嵌入向量的微调方法，不同于硬提示学习修改离散的输入 Tokens，软提示学习利用可学习的参数向量进行微调，能够实现更高效准确的提示优化。

Soft Prompt Tuning 也称为 Prompt Learning，核心理念是在嵌入查询 Token 之前预置一个可训练的参数向量，能够通过梯度下降等优化算法在特定数据集上训练以提高模型性能。

CoOP (Conditional Prompt Learning) 是最早提出提示学习概念的方法之一，通过软提示学习提示词以微调 CLIP 模型在下游任务上的表现。

```

1 x = EmbeddingLayer(input_ids)
2 x = concatenate([soft_prompt_tensor, x],
3                 dim=seq_len)
4 output = model(x)
  
```

Prefix Tuning 前缀微调（Prefix Tuning）类似提示学习，在每一个 Transformer 块中添加可学习的嵌入向量，而不仅仅是嵌入输入，这样保证了训练的平滑性。

```

1 def transformer_block_with_prefix(x):
2     # 1:
3     soft_prompt = FullyConnectedLayers( # Prefix
4                                         soft_prompt) # Prefix
5     # 2:
  
```

```

6  x = concatenate([soft_prompt, x],    # Prefix
7                  dim=seq_len)         # Prefix
8  # 3:
9  x = SelfAttention(x)
10 x = LayerNorm(x + residual)
11 residual = x
12 x = FullyConnectedLayers(x)
13 x = LayerNorm(x + residual)
14 return x

```

如图1.16所示，在传递给 Transformer 的输入中，通过全连接层提取 Soft Prompt 的特征嵌入，然后与原始输入特征嵌入进行拼接，以实现多级 Transformer 的调节。

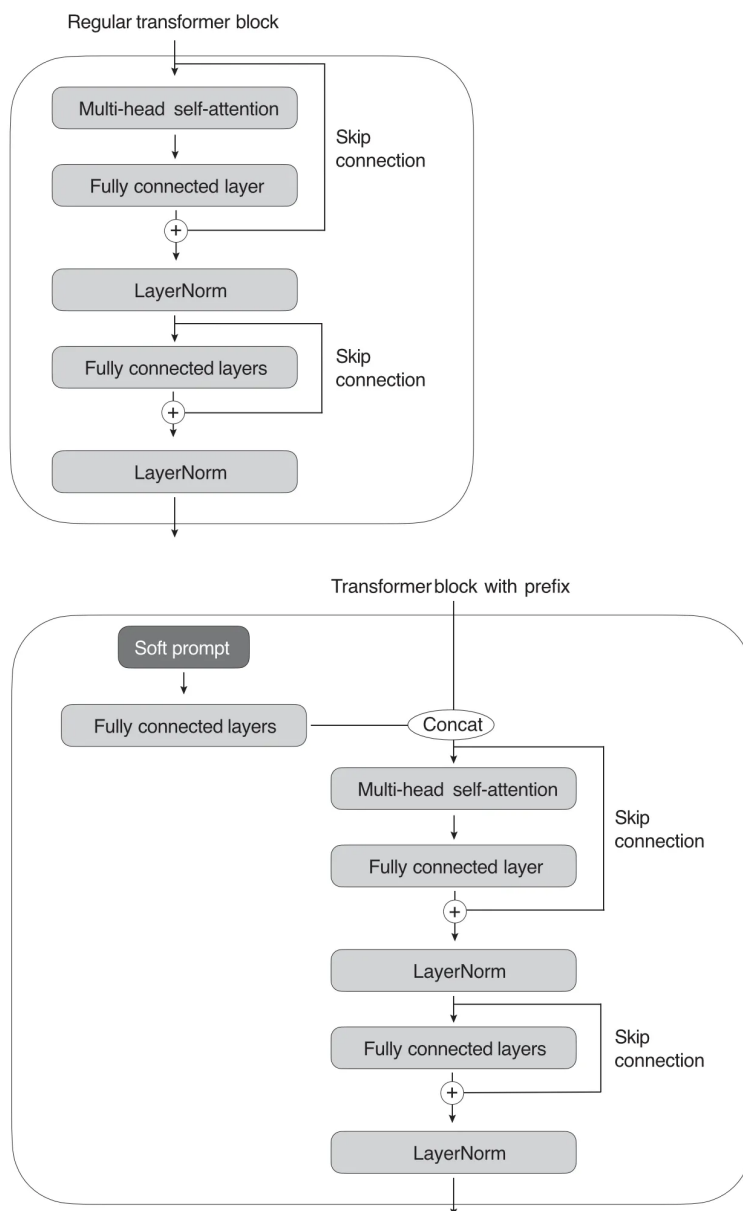


Figure 1.16: 采用 Prefix Tuning 的 Transformer 结构对比 (Raschka 2024b)

Adapter Methods 相比于 Prefix Tuning 方法, Adapter Methods 通过在 Transformer 层添加额外的参数 (全连接层) 进行学习, 额外的全连接层是被添加在多头注意力机制以及原有的全连接层之后, 实现对每一个 Transformer 的微调。

如图1.17所示, 在微调大模型时, 只有新添加的 Adapter 层参数进行更新, 其他的 Transformers 层保持不变, Adapter 一般采用很小的结构, 两层全连接层, 第一层将特征映射到低维空间中进行压缩, 第二层将特征映射回原始高维空间中, 因此是非常高效的。

```

1 def transformer_block_with_adapter(x):
2     residual = x
3     x = SelfAttention(x)
4     x = FullyConnectedLayers(x) # Adapter
5     x = LayerNorm(x + residual)
6     residual = x
7     x = FullyConnectedLayers(x)
8     x = FullyConnectedLayers(x) # Adapter
9     x = LayerNorm(x + residual)
10    return x

```

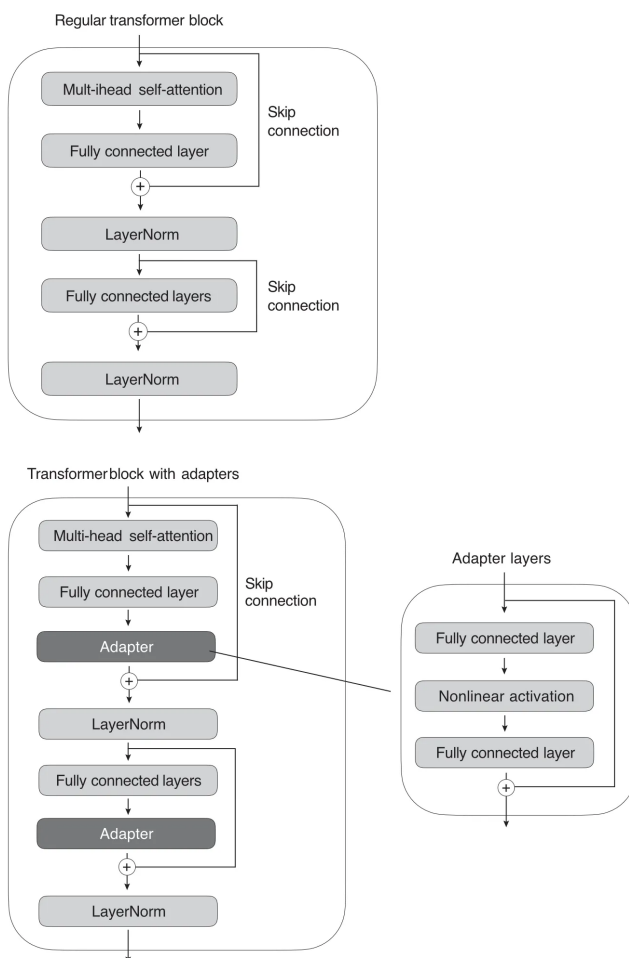


Figure 1.17: 采用 Adapter 的 Transformer 结构对比 (Raschka 2024b)

Low-rank Adaptation 低秩适应 (Low-rank Adaptation, LoRA) 指的是利用低秩变换重新参数化预训练 LLM 权重, 从而实现模型在特定任务上的微调。低秩变换利用多个低维表示来估计一个高维矩阵,

其中低维变换（低秩估计）通过找到能够有效捕获原始数据中大部分信息的更少的维度组合来实现，例如主成分分析（PCA）和奇异值分解（SVD）。

假设预训练 LLM 的权重更新矩阵为 $\Delta W \in \mathbb{R}_{A \times B}$ ，低秩分解将权重更新矩阵分解为两个更小的矩阵的乘积，即

$$\Delta W = W_A W_B.$$

其中， $W_A \in \mathbb{R}^{A \times h}$, $W_B \in \mathbb{R}^{h \times B}$ ， h 是低秩变换的秩，通常远小于 A 和 B 。

W 的参数量是 $A \times B$ ，而 W_A 和 W_B 的参数量是 $A \times h + h \times B$ ，因此

$$\begin{aligned} A \times B &> A \times h + B \times h \\ &= (A + B) \times h. \\ \Rightarrow h &< \frac{A \times B}{A + B} \\ &< \frac{AB}{2\sqrt{AB}} = \frac{\sqrt{AB}}{2}. \end{aligned}$$

如果 $A = B = 25$ 且 $h = 10$ ，则降低的参数量为： $25^2 - 2 \times 25 \times 10 = 625$ ，可见参数学习还是非常高效的。

通过保持预训练模型权重不变，只训练新权重更新矩阵 W_A 和 W_B ，可以显著减少微调模型的参数量和计算复杂性。

如下代码所示，原有的矩阵相乘是模型的输出，再加上低秩变换更新权重矩阵，得到新的输出。其中，`scalar` 表示缩放因子，用于平衡两部分结合结果。这种方式平衡了预训练模型的知识与新的特定任务的适应。

```
1 def lora_forward_matmul(x):
2     h = x . W # Regular matrix multiplication
3     h += x . (W_A . W_B) * scalar
4     return h
```

8 July 30

8.1 Inequality

不等式规则可以借助缩放定理，实现上下界推导、范围缩放等。

平方不等式 $A^2 + B^2 \geq 2AB, A + B > 2\sqrt{AB} (A \geq 0, B \geq 0)$

Appendix A

Formulas

1 Gaussian distribution

Definition A.1 (Gaussian distribution). *Gaussian distribution*

Theorem A.1 (Central limit theorem).

Bibliography

- Bavishi, Rohan (2023). *Fuyu-8B: A Multimodal Architecture for AI Agents* (cit. on p. 17).
- Chopra, S. et al. (2005). *Learning a similarity metric discriminatively, with application to face verification*. 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05). Vol. 1, 539–546 vol. 1 (cit. on p. 10).
- Du, Nan et al. (2022). *GLaM: Efficient Scaling of Language Models with Mixture-of-Experts*. arXiv: 2112.06905 [cs.CL] (cit. on p. 3).
- Raschka, Sebastian (2024a). *Understanding Multimodal LLMs* (cit. on pp. 13–17).
- (2024b). *Using and Finetuning Pretrained Transformers* (cit. on pp. 18, 19, 21, 22).
- Schroff, Florian et al. (June 2015). *FaceNet: A unified embedding for face recognition and clustering*. 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). IEEE, pp. 815–823 (cit. on pp. 10, 11).
- Sucheng, Ren et al. (2025). *Beyond Next-Token: Next-X Prediction for Autoregressive Visual Generation*. arXiv preprint arXiv:2502.20388 (cit. on pp. 11, 12).

Alphabetical Index

G

Gaussian distribution [24](#)